

# Viva Preparation - Member 2 - Event Service

---

## Owned Component

Owned microservice:

- `event-service`

Main files to show:

- `services/event-service/src/app.js`
- `services/event-service/src/controllers/eventController.js`
- `services/event-service/src/routes/eventRoutes.js`
- `services/event-service/src/models/Event.js`
- `services/event-service/Dockerfile`
- `sonar/event-service.properties`

## What To Say

I implemented the event management service. This service is responsible for event creation, event listing, event updates, publishing control, and seat availability management.

The event service is important because the booking service depends on it before creating a booking. A booking should only be created if the event exists and has enough available seats.

## Endpoints To Explain

- `GET /api/events` - lists events.
- `GET /api/events/:id` - returns event details.
- `POST /api/events` - creates an event.
- `PUT /api/events/:id` - updates an event.
- `DELETE /api/events/:id` - deletes an event.
- `PATCH /api/events/:id/reserve-seats` - reserves seats for a booking.
- `PATCH /api/events/:id/release-seats` - releases seats after rejection or cancellation.

Through the gateway these become:

- `/events`
- `/events/:id`
- `/events/:id/reserve-seats`
- `/events/:id/release-seats`

## Integration Point

Integration to demonstrate:

`booking-service` calls `event-service` during the booking flow.

Example explanation:

When a customer requests a booking, the booking service calls the event service to check event details and reserve seats. If the booking is rejected or cancelled, the booking service calls the event service again to release those seats.

This proves inter-service communication between two members' services.

## Demo Steps

1. Log in as an organizer.
2. Create an event.
3. Publish the event if the UI supports publishing.
4. Log in as a customer.
5. Request a booking for that event.
6. Explain that the booking request triggers a call from booking-service to event-service.
7. Show that seats are managed through reserve/release logic.

## Security Points

- Event creation and management are protected by JWT.
- Organizer/admin roles control event management operations.
- Seat reservation is not handled directly by the frontend; it is performed through backend service logic.
- The deployed event service uses internal ingress in Azure and is reached through the gateway or service-to-service calls.
- Secrets and database connection strings are environment-based.

## DevOps And Cloud Points

- The service has its own Dockerfile.
- The Docker image is published to GHCR.
- The service is deployed to Azure Container Apps.
- The service has a separate SonarCloud project:
  - `EventHub Event Service`
- The GitHub Actions workflow scans this service using:
  - `sonar/event-service.properties`

## Likely Viva Questions

What is the purpose of the event service?

It manages event data such as title, venue, date, category, price, total seats, available seats, and publish status.

How does your service communicate with another microservice?

The booking service calls event-service endpoints to check event information, reserve seats, and release seats.

Why is seat reservation in the event service?

The event service owns event data and seat availability. Keeping seat changes inside this service protects data consistency and prevents other services from directly editing event records.

## What happens if there are not enough seats?

The event service rejects the reservation request. Then the booking service should not create a successful booking.

## Why use an API gateway?

The frontend has one backend base URL. The gateway hides internal service URLs and routes `/events` requests to the event service.

## How is your service deployed?

It is containerized with Docker, published as an image, and deployed to Azure Container Apps.

## How did you apply DevSecOps?

The service is scanned by SonarCloud through GitHub Actions. This checks code quality, maintainability, bugs, vulnerabilities, and code smells.

# Common Group Questions

## What is the application?

EventHub is a microservice-based event booking platform. Customers can browse events and request bookings, organizers can manage events and approvals, and admins can oversee activity.

## What are the four microservices?

- Auth Service
- Event Service
- Booking Service
- Notification Service

## What is the API gateway used for?

The gateway is the public backend entry point. The frontend calls the gateway, and the gateway routes requests to the correct internal service.

## What is the main integration flow?

The strongest flow is booking:

1. Customer requests a booking.
2. Booking service calls event service to check and reserve seats.
3. Booking service stores a pending booking.
4. Booking service calls notification service to create alerts.
5. Booking service calls auth service when admin user data is needed.

## What DevOps practices are implemented?

- GitHub repository

- GitHub Actions CI
- Docker containerization
- GHCR container image publishing
- Azure Container Apps deployment
- SonarCloud DevSecOps scanning

What security practices are implemented?

- JWT authentication
- Role-based access
- Internal microservice ingress in Azure
- Environment variables and GitHub secrets
- SonarCloud static analysis
- Express Helmet middleware

## Quick Emergency Checks

Gateway health:

```
curl https://gateway.mangocoast-efcefaea.southeastasia.azurecontainerapps.io
```

Events route:

```
curl https://gateway.mangocoast-efcefaea.southeastasia.azurecontainerapps.io/events
```

Event logs:

```
az containerapp logs show --name event-service --resource-group eventhub-rg --tail 100
```